

13. 함수형 프로그래밍

#0.강의/1.자바로드맵/7.자바-고급3편

- /프로그래밍 패러다임
- /함수형 프로그래밍이란?
- /자바와 함수형 프로그래밍1
- /자바와 함수형 프로그래밍2
- /자바와 함수형 프로그래밍3
- /정리

프로그래밍 패러다임

프로그래밍 패러다임이란 프로그램을 구성하고 구현하는 사상이나 접근법을 말한다.

대표적인 프로그래밍 패러다임은 크게 다음과 같이 분류할 수 있다.

- **명령형 프로그래밍 (Imperative)**
 - 절차지향 프로그래밍 (Procedural)
 - 객체지향 프로그래밍 (OOP)
- **선언형 프로그래밍 (Declarative)**
 - 함수형 프로그래밍 (Functional)

명령형 프로그래밍(Imperative Programming)

- 프로그램이 **어떻게(How)** 동작해야 하는지 세세한 제어 흐름을 통해 기술
- 대표적인 하위 스타일로 **절차지향(Procedural)**과 **객체지향(OOP)**을 포함
 - **절차지향**은 프로시저, 함수를 기반으로 로직을 절차적으로 구성
 - **객체지향**은 데이터(필드)와 함수(메서드)를 하나로 묶은 객체를 중심으로 설계

선언형 프로그래밍(Declarative Programming)

- **무엇(What)**을 해야 하는지에 초점을 맞추어, 목적만 선언하고 구현 방식은 추상화
- 대표적인 예로 **함수형 프로그래밍(Functional)**, **SQL**, **HTML** 등이 있음
 - **함수형 프로그래밍**은 순수 함수를 조합하며, 부수 효과와 가변 상태를 최소화하여 로직을 표현

명령형 프로그래밍 (Imperative Programming)

- **핵심 개념**: 어떻게(How) 할 것인지 구체적으로 명령(Instruction)을 내리는 방식

- **특징**
 - 프로그램이 어떤 순서와 단계로 동작해야 하는지를 구체적인 제어 흐름(조건문, 반복문 등)으로 기술
 - 변수의 값이 바뀌면서 상태(state)가 변해감
 - CPU의 동작 방식(메모리 수정, 제어 흐름에 따른 실행)과 유사하여, 전통적인 하드웨어와의 직관적인 일치
 - 예시: C, C++, Java 등 대부분의 언어가 명령형 특성을 지님
- **장단점**
 - 장점: 컴퓨터의 동작 방식과 매우 유사해 이해하기 직관적, 제어 흐름을 상세히 제어하기 쉽다.
 - 단점: 프로그램 규모가 커지면 상태 변경에 따른 복잡도가 증가

절차지향 프로그래밍 (Procedural Programming)

- **핵심 개념:** 명령형 프로그래밍의 대표적인 형태로, 프로그램을 절차(Procedure)나 함수(Function) 단위로 나누어 순서대로 실행
- **특징**
 - 명령형 패러다임의 하위 개념으로 볼 수 있음
 - 공통된 로직을 재사용하기 위해 함수나 프로시저를 만들어 사용
 - "데이터와 절차가 분리되어 있다"라는 말로도 자주 설명됨. 즉, 함수(절차)는 별도로 정의해 두고, 여러 데이터에 대해 같은 절차를 적용
 - 예시: C, Pascal 등
- **장단점**
 - 장점: 구조적 프로그래밍 기법(모듈화, 함수화)으로 코드 가독성 상승, 코드 재사용성 향상
 - 단점: 데이터와 로직이 명확히 분리되지 않을 때, 코드 유지 보수가 어렵고 대형 프로젝트에서 복잡성 증가

참고 - 함수와 프로시저

- **프로시저(Procedure):** 일련의 명령문들을 하나의 단위로 묶은 것으로, 특정 작업이나 행동을 수행하는 데 중점을 둔다. 프로시저는 반드시 값을 반환할 필요가 없으며, 주로 상태 변경이나 특정 동작 수행에 초점을 맞춘다.
 - 자바로 쉽게 비유를 하자면 `void`를 반환하는 메서드를 떠올리면 된다.
- **함수(Function):** 수학적 함수의 개념에서 유래했으며, 입력값을 받아 처리하고 결과값을 반환하는 것이 주 목적이다. 함수는 보통 값을 계산하고 반환하는 데 중점을 둔다.

객체지향 프로그래밍 (Object-Oriented Programming)

- **핵심 개념:** 프로그램을 객체(Object)라는 추상화된 단위로 구성. 각 객체는 상태(필드, 속성)와 행동(메서드)을 갖고 있으며, 메시지 교환(메서드 호출)을 통해 상호작용
- **특징**
 - 캡슐화(Encapsulation), 추상화(Abstraction), 상속(Inheritance), 다형성(Polymorphism)과 같은 특

징이 있음

- 데이터와 해당 데이터를 처리하는 함수를 하나의 객체로 묶어서 관리해 유지보수성과 확장성을 높인다.
- 예시: Java, C++, C#

- **장단점**

- 장점: 객체라는 단위로 묶이므로 코드 재사용성, 확장성, 유지보수성 우수. 대규모 시스템 설계에 적합
- 단점: 과도한 객체 분리나 복잡한 상속 구조 등으로 인해 오히려 복잡도가 증가할 수 있음

선언형 프로그래밍 (Declarative Programming)

- **핵심 개념:** "무엇을(What) 할 것인지"를 기술하고, "어떻게(How)" 구현, 실행될지는 위임하는 방식
- **특징**
 - 구체적인 제어 흐름(조건문, 반복문 등)을 직접 작성하기보다, 원하는 결과나 조건을 선언적으로 표현
 - 상태 변화보다는 결과에 초점을 맞추어 코드를 작성
 - 대표적인 예시: SQL(쿼리로 원하는 데이터나 조건을 선언), HTML(화면 구조/콘텐츠만 기술하면 브라우저가 렌더링)
 - 함수형 프로그래밍 등이 선언형 패러다임에 속하거나 밀접하게 관련됨

장단점

- 장점: 구현의 복잡한 로직을 많이 숨길 수 있어, 높은 수준에서 문제 해결에 집중 가능. 비즈니스 로직을 직관적으로 표현하기 쉬움
- 단점: 언어나 환경이 제공하는 추상화 수준에 의존적이며, 내부 동작이 보이지 않을 경우 디버깅이 어려울 수 있음. 낮은 수준의 최적화나 세밀한 제어가 필요한 상황에서는 제약이 생길 수도 있음

함수형 프로그래밍 (Functional Programming)

- **핵심 개념:** 무엇(What)을 할 것인지를 수학적 함수(Function)들로 구성하고, 부수 효과(Side Effect) 최소화 및 불변성(Immutable State)을 강조하는 프로그래밍 방식
- **특징**
 - 선언형(Declarative) 접근에 가까움: "어떻게"가 아니라, "어떤 결과"를 원한다고 선언
 - 순수 함수(Pure Function)를 중시: 같은 입력이 주어진다면 항상 같은 출력
 - 데이터는 불변(Immutable)하게 처리: 재할당 대신 새로운 데이터를 만들어 반환
 - 함수가 일급 시민(First-Class Citizen)으로 취급: 고차 함수(Higher-Order Function), 함수를 인자로 넘기거나 반환 가능
 - 예시: Haskell, Clojure, Scala, Java(람다와 함수형 인터페이스를 통한 부분 지원)

장단점

- 장점: 상태 변화가 없거나 최소화되므로 디버깅과 테스트 용이, 병렬 처리 및 동시성 처리가 간단해지는 경향
- 단점: 명령형 사고방식에 익숙한 프로그래머에게는 초기 접근이 어려울 수 있음, 계산 과정에서의 메모리 사용이 증가할 수 있음

정리

- **명령형 프로그래밍:** 컴퓨터가 실행할 단계별 명령을 직접 제어하고 기술
 - **절차지향 프로그래밍:** 명령형 패러다임 안에서 프로그램을 함수(프로시저) 단위로 분할하여 개발
 - **객체지향 프로그래밍:** 데이터와 함수를 객체라는 단위로 묶어 추상화하고 상호 작용
- **선언형 프로그래밍:** 무엇(What)을 해야 하는지에 집중하여, 구체적인 구현 방식이나 절차 흐름을 추상화하고 선언적으로 기술
 - **함수형 프로그래밍:** 순수 함수와 불변성에 기반하여, 선언적으로 로직을 기술

이러한 패러다임은 서로 대립되지 않으며, 문제 상황이나 설계 목표에 따라 적절히 선택하고 조합해 사용할 수 있다. 실제로 대부분의 언어들은 여러 패러다임을 부분적으로 섞어서 지원(멀티 패러다임)하므로, 상황에 맞추어 다른 스타일을 병행해 쓰는 능력이 중요하다. 쉽게 이야기해서 한 프로젝트 안에서도 특정 부분은 절차지향으로 특정 부분은 객체지향으로 특정 부분은 함수형 스타일을 사용한다. 따라서 문제를 해결하기 가장 나은 방법을 각각 고민하고 선택해야 한다.

자바 언어의 패러다임

자바는 객체지향 프로그래밍 언어이다. 그렇다고 해서 절차지향 방식을 전혀 사용하지 않는 것은 아니다.

여기서 **지향**이라는 단어가 중요하다.

무언가를 지향한다는 것은 쉽게 이야기해서 **그 방향을 주된 목표나 철학으로 삼고, 그에 맞춰 설계와 구현을 이끌어가는 것**을 의미한다. 예를 들어 자바가 '객체지향 프로그래밍'을 지향한다는 말은, 자바가 클래스와 객체라는 개념을 중심으로 언어적 기능들을 제공하고, 개발자들에게 객체지향적인 설계를 권장하도록 만들어졌다는 뜻이다.

하지만 '지향'한다고 해서 오직 그 방법론만을 고집해야 한다는 의미는 아니다. 자바가 객체지향 패러다임에 충실한 언어이지만, 필요에 따라 절차지향적인 방식(예: `static` 메서드 활용)으로 코드를 작성할 수도 있다.

예를 들어, 자바에서도 `main` 메서드 하나만 두고 여러 개의 `static` 메서드를 호출하는 방식으로 코드를 구성한다면 절차지향적인 접근을 취할 수 있다. 이러한 방식은 전역 변수 대신 클래스 변수를 활용하거나, 필요한 변수를 메서드 내부에서만 유지함으로써 유지보수 부담을 어느 정도 줄일 수 있지만, 여전히 '상태 변화'와 '명령의 순서'에 집중한다는 점에서 절차지향 프로그래밍과 유사하다.

즉, 자바는 객체지향 언어이지만, 동시에 절차지향 프로그래밍의 특징을 일부 차용하여 유연하게 코드를 작성할 수 있다. 이것은 곧 자바의 주요 장점 중 하나이기도 하다. 실제로 많은 자바 개발자들은 규모가 작은 기능이나 간단한 처리를 할 때 절차지향적인 관점을 이용하고, 복잡한 시스템이나 대규모 프로젝트에서는 객체지향적인 모델링 기법을 적극 활용한다. 이처럼 하나의 언어 안에서 다양한 패러다임을 혼합해 사용함으로써, 개발자는 문제 상황에 맞는 최적의 방법론

을 선택할 수 있다.

또한 자바 8 이후로는 함수형 프로그래밍 스타일의 코드(람다, 스트림 API)를 지원하여, 상황에 맞는 다양한 패러다임을 혼합해 적용할 수 있도록 유연성을 높였다.

결국, '객체지향 프로그래밍 언어'라 함은 그 언어의 기본 구조와 철학이 객체지향 개념(클래스, 객체, 캡슐화, 다형성 등)을 중심으로 설계되어 있다는 것을 강조하는 표현이지, 그 언어가 다른 모든 패러다임을 전면 배제한다는 의미는 아니다.

함수형 프로그래밍이란?

함수형 프로그래밍(Functional Programming)은 프로그램을 함수(수학적 함수)를 조합해 만드는 방식에 초점을 두는 프로그래밍 패러다임이다. 명령형 프로그래밍(Imperative Programming)처럼 **어떻게(How)** 할 것인지(절차와 상태 변화를 명시)보다는, 필요한 결과를 얻기 위해 **무엇(What)**을 계산할 것인가를 강조한다. 함수를 일급 시민(First-class Citizen)으로 취급하고, 불변(Immutable) 상태를 지향하며, 순수 함수(Pure Function)를 중심에 두는 것이 주요 특징이다.

함수형 프로그래밍의 핵심 개념과 특징

- **순수 함수(Pure Function)**
 - 같은 인자를 주면 항상 같은 결과를 반환하는 함수이다.
 - 외부 상태(변할 수 있는 전역 변수 등)에 의존하거나, 외부 상태를 변경하는 부수 효과(Side Effect)가 없는 함수를 의미한다.
- **부수 효과(Side Effect) 최소화**
 - 함수형 프로그래밍에서는 상태 변화를 최소화하기 위해 변수나 객체를 변경하는 것을 지양한다.
 - 이로 인해 프로그램의 버그가 줄어들고, 테스트나 병렬 처리(Parallelism), 동시성(Concurrency) 지원이 용이해진다.
- **불변성(Immutable State) 지향**
 - 데이터는 생성된 후 가능한 한 변경하지 않고, 변경이 필요한 경우 새로운 값을 생성해 사용한다.
 - 가변 데이터 구조에서 발생할 수 있는 오류를 줄이고, 프로그램의 예측 가능성을 높여준다.
- **일급 시민(First-class Citizen) 함수**
 - 함수가 일반 값(숫자, 문자열, 객체(자료구조) 등)과 동일한 지위를 가진다.
 - 함수를 변수에 대입하거나, 다른 함수의 인자로 전달하거나, 함수에서 함수를 반환하는 고차 함수(Higher-order Function)를 자유롭게 사용할 수 있다.
- **선언형(Declarative) 접근**
 - **어떻게**가 아닌 **무엇**을 계산할지 기술한다.
 - 복잡한 제어 구조나 상태 관리를 함수의 합성과 함수 호출로 대체하여 간결하고 가독성 높은 코드를 작성한

다.

- **함수 합성(Composition)**

- 간단한 함수를 조합해 더 복잡한 함수를 만드는 것을 권장한다.
- 작은 단위의 함수들을 체이닝(Chaining)하거나 파이프라이닝(Pipelining)해서 재사용성을 높이고, 코드 이해도를 높인다.

- **Lazy Evaluation(지연 평가) (선택적 특징)**

- 필요한 시점까지 계산을 미루는 평가 전략이다.
- 불필요한 계산 비용을 줄인다.

함수형 프로그래밍의 장점

- **코드 가독성 및 유지보수성**: 순수 함수, 불변 데이터 구조를 활용하면, 프로그램 흐름이 명료해진다.
- **병렬 처리 및 동시성**: 데이터가 불변이므로, 여러 스레드가 같은 데이터를 다룰 때 충돌이 적어 병렬 실행이 유리하다.
- **테스트 용이성**: 순수 함수는 외부 환경에 의존하지 않으므로, 테스트가 간단하다.

함수형 프로그래밍의 단점

- **학습 곡선**: 기존 명령형/객체지향 스타일에 익숙한 개발자에게는, 순수 함수, 고차 함수 등 함수형 개념이 처음에는 이해하기 쉽지 않다.
- **성능**: 모든 데이터를 불변하게 다루는 경우, 매번 새로운 데이터를 생성해야 하므로 메모리 사용량이 늘어날 수 있다. (다만, 언어나 런타임 레벨에서 최적화를 제공할 수 있다.)

주요 함수형 프로그래밍 언어 및 활용

대표적인 순수 함수형 프로그래밍 언어로는 Haskell이 있다. Java, JavaScript, Python 등의 전통적인 언어들은 순수한 함수형 프로그래밍 언어는 아니지만, 람다 표현식, 고차 함수 등 함수형 스타일을 점진적으로 지원함으로써, 함수형 프로그래밍의 장점을 활용한다.

정리하자면, 함수형 프로그래밍이란 **순수 함수, 불변성, 부수 효과 최소화**를 핵심으로 하여, **함수를 일급 시민으로 취급**하고, **합성(Composition)**을 통해 문제를 해결하는 프로그래밍 패러다임이다. 이러한 접근 방식은 프로그램을 이해하기 쉽게 만들고, 오류나 동시성 문제를 줄여 유지보수와 확장이 용이한 코드를 작성하도록 돕는다.

자바는 **명령형, 객체지향**이 주된 패러다임이고, 거기에 **람다 등 함수형 문법**이 일부 도입된 "멀티 패러다임(Multi-paradigm) 언어"이다. 따라서 자바는 객체지향 중심이지만, 부분적으로 함수형 특성을 지원하는 언어 정도로 이해하면 된다.

자바와 함수형 프로그래밍1

함수형 프로그래밍은 다음과 같은 특징이 있다.

1. 순수 함수(Pure Function)
2. 부수 효과(Side Effect) 최소화
3. 불변성(Immutable State) 지향
4. 일급 시민(First-class Citizen) 함수
5. 선언형(Declarative) 접근
6. 함수 합성(Composition)
7. Lazy Evaluation(지연 평가) (선택적 특징)

자바는 람다와 함수형 인터페이스를 통해 함수형 프로그래밍의 장점을 자바 언어로 가져왔다.

예제 코드를 통해 함수형 프로그래밍의 특징을 하나씩 정리해보자.

순수 함수(Pure Function)

- 같은 인자를 주면 항상 같은 결과를 반환하는 함수이다.
- 외부 상태(변할 수 있는 전역 변수 등)에 의존하거나, 외부 상태를 변경하는 부수 효과(Side Effect)가 없는 함수를 의미한다.

순수 함수 예제1

```
package functional;

import java.util.function.Function;

public class PureFunctionMain1 {

    public static void main(String[] args) {
        Function<Integer, Integer> func = x -> x * 2;
        System.out.println("result1 = " + func.apply(10));
        System.out.println("result2 = " + func.apply(10));
    }

}
```

실행 결과

```
result1 = 20
result2 = 20
```

이 예제에서는 해당 함수가 **입력값이 동일하면 항상 동일한 결과를 반환**한다는 점을 보여준다. 외부 상태에 의존하거나 수정하지 않으므로, 10을 두 번 호출해도 결과가 모두 20으로 동일하다. 이는 순수 함수의 중요한 특징이다.

부수 효과(Side Effect) 최소화

- 함수형 프로그래밍에서는 상태 변화를 최소화하기 위해 변수나 객체를 변경하는 것을 지양한다.
- 이로 인해 프로그램의 버그가 줄어들고, 테스트나 병렬 처리(Parallelism), 동시성(Concurrency) 지원이 용이해진다.

부수 효과 예제1

```
package functional;

import java.util.function.Function;

public class SideEffectMain1 {

    public static int count = 0;

    public static void main(String[] args) {
        System.out.println("before count = " + count);

        Function<Integer, Integer> func = x -> {
            count++; // 부수 효과(Side Effect)
            return x * 2;
        };
        func.apply(10);
        System.out.println("after count = " + count);
    }
}
```



```
before count = 0
after count = 1
```

이 예제에서는 함수가 `count` 라는 전역 변수를 변경하고 있다. 함수 호출 전후로 `count` 가 0에서 1로 바뀌는데, 이런 **외부 상태 변화**가 부수 효과의 대표적인 예시이다. 함수형 프로그래밍에서는 이러한 외부 상태 변경을 최소화하는 것을 권장한다.

부수 효과 예제2

```
package functional;

import java.util.function.Function;

public class SideEffectMain2 {

    public static void main(String[] args) {
        Function<Integer, Integer> func = (x) -> {
            int result = x * 2;

            // 부수 효과(Side Effect)
            System.out.println("x = " + x + ", result = " + (x * 2));
            return result;
        };
        func.apply(10);
    }
}
```

실행 결과

```
x = 10, result = 20
```

이 예제에서는 **콘솔에 출력을 하는 동작**이 부수 효과이다. 출력은 결과값 자체와 무관하지만, 외부 세계(콘솔)에 영향을 미치므로 순수 함수로 보기는 어렵다.

부수 효과 예제3

```
package functional;

import java.util.function.Function;

public class SideEffectMain3 {

    public static void main(String[] args) {
        Function<Integer, Integer> func = x -> x * 2;
        int x = 10;
        Integer result = func.apply(10);
        // 부수 효과는 순수 함수와 분리해서 실행
        // 출력은 별도로 처리해 순수성을 유지
        System.out.println("x = " + x + ", result = " + result);
    }

}
```

실행 결과

```
x = 10, result = 20
```

이 예제에서는 연산을 담당하는 함수(func)가 외부 상태를 전혀 수정하지 않는 순수 함수이고, 출력(부수 효과)은 필요한 시점에만 별도로 수행한다. 이렇게 연산(순수 함수)과 외부 동작(부수 효과)을 명확히 분리함으로써 순수 함수를 유지할 수 있다.

부수 효과 컬렉션 예제

```
package functional;

import java.util.ArrayList;
import java.util.List;

public class SideEffectListMain {

    public static void main(String[] args) {
```

```

List<String> list1 = new ArrayList<>();
list1.add("apple");
list1.add("banana");

System.out.println("before list1 = " + list1);
changeList1(list1);
System.out.println("after list1 = " + list1);

List<String> list2 = new ArrayList<>();
list2.add("apple");
list2.add("banana");
System.out.println("before list2 = " + list2);
List<String> result = changeList2(list2);
System.out.println("after list2 = " + list2);
System.out.println("result = " + result);
}

private static void changeList1(List<String> list) {
    for (int i = 0; i < list.size(); i++) {
        list.set(i, list.get(i) + "_complete");
    }
}

private static List<String> changeList2(List<String> list) {
    List<String> newList = new ArrayList<>();
    for (String s : list) {
        newList.add(s + "_complete");
    }
    return newList;
}
}

```

실행 결과

```

before list1 = [apple, banana]
after list1 = [apple_complete, banana_complete]
before list2 = [apple, banana]
after list2 = [apple, banana]
result = [apple_complete, banana_complete]

```

- `changeList1` 함수는 리스트 원본을 직접 변경함으로써 부수 효과를 일으킨다.

- 반면 `changeList2` 함수는 새로운 리스트를 생성해서 반환함으로써, 원본 리스트를 변경하지 않는다.
- 함수형 프로그래밍에서는 `changeList2` 와 같은 방식이 권장된다.

자바와 함수형 프로그래밍2

불변성(Immutable State) 지향

- 데이터는 생성된 후 가능한 한 변경하지 않고, 변경이 필요한 경우 새로운 값을 생성해서 사용한다.
- 가변 데이터 구조에서 발생할 수 있는 오류를 줄이고, 프로그램의 예측 가능성을 높여준다.
- 불변성은 데이터를 변경하지 않기 때문에 부수 효과와도 관련이 있다.

값을 변경할 수 있는 객체

```
package functional;

public final class MutablePerson {
    private String name;
    private int age;

    public MutablePerson(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

```

@Override
public String toString() {
    return "MutablePerson{" +
        "name='" + name + '\'' +
        ", age=" + age +
        '}';
}
}

```

- MutablePerson은 **setter** 메서드로 인해 객체 생성 후에도 상태(필드 값)를 언제든지 변경할 수 있다.

값을 변경할 수 없는 객체

```

package functional;

public final class ImmutablePerson {
    private final String name;
    private final int age;

    public ImmutablePerson(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Getter만 제공, Setter 없음
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    // 변경이 필요한 경우, 기존 객체를 수정하지 않고 새 객체를 반환
    public ImmutablePerson withAge(int newAge) {
        return new ImmutablePerson(name, newAge);
    }

    @Override
    public String toString() {
        return "ImmutablePerson{" +
            "name='" + name + '\'' +

```

```

        ", age=" + age +
        '}';
    }
}

```

- `ImmutablePerson`은 필드가 **final**이며, 생성 후에는 상태를 변경할 수 없도록 설계되었다. 나이를 바꿀 때도 `withAge`로 새 객체를 생성하므로, 원본 객체는 변하지 않고 부수 효과가 최소화된다.

불변성 지향 예제1

```

package functional;

public class ImmutableMain1 {
    public static void main(String[] args) {
        MutablePerson m1 = new MutablePerson("Kim", 10);
        MutablePerson m2 = m1;
        m2.setAge(11);
        System.out.println("m1 = " + m1);
        System.out.println("m2 = " + m2);

        ImmutablePerson i1 = new ImmutablePerson("Lee", 20);
        ImmutablePerson i2 = i1.withAge(21); // i2는 새로운 객체
        System.out.println("i1 = " + i1); // 20
        System.out.println("i2 = " + i2); // 21
    }
}

```

실행 결과

```

m1 = MutablePerson{name='Kim', age=11}
m2 = MutablePerson{name='Kim', age=11}
i1 = ImmutablePerson{name='Lee', age=20}
i2 = ImmutablePerson{name='Lee', age=21}

```

`m1`과 `m2`가 동일 객체를 참조하므로, 하나를 변경하면 원본도 변화한다. 반면 `ImmutablePerson`은 `withAge`호출 시 새로운 객체를 생성하기 때문에, `i1`은 원본 그대로 유지되고 `i2`만 변경된 `age` 값을 가진다.

불변성 지향 예제2

```

package functional;

import java.util.List;

public class ImmutableMain2 {
    public static void main(String[] args) {
        MutablePerson m1 = new MutablePerson("Kim", 10);
        MutablePerson m2 = new MutablePerson("Lee", 20);

        List<MutablePerson> originList = List.of(m1, m2);
        System.out.println("originList = " + originList);
        List<MutablePerson> resultList = originList.stream()
            .map(p -> {
                p.setAge(p.getAge() + 1);
                return p;
            })
            .toList();

        System.out.println("=== 실행 후 ===");
        System.out.println("originList = " + originList);
        System.out.println("resultList = " + resultList);
    }
}

```

실행 결과

```

originList = [MutablePerson{name='Kim', age=10}, MutablePerson{name='Lee',
age=20}]
=== 실행 후 ===
originList = [MutablePerson{name='Kim', age=11}, MutablePerson{name='Lee',
age=21}]
resultList = [MutablePerson{name='Kim', age=11}, MutablePerson{name='Lee',
age=21}]

```

stream에서 setAge를 사용하면서, **원본 리스트(originList)의 데이터까지 변경되어 버렸다**. 가변(Mutable) 상태는 이런 식으로 예상치 못한 곳에서 영향을 미치므로, 버그나 유지보수 어려움을 초래할 수 있다.

불변성 지향 예제3

```

package functional;

import java.util.List;

public class ImmutableMain3 {
    public static void main(String[] args) {
        ImmutablePerson i1 = new ImmutablePerson("Kim", 10);
        ImmutablePerson i2 = new ImmutablePerson("Lee", 20);

        List<ImmutablePerson> originList = List.of(i1, i2);
        System.out.println("originList = " + originList);
        List<ImmutablePerson> resultList = originList.stream()
            .map(p -> new ImmutablePerson(p.getName(), p.getAge() + 1))
            .toList();

        System.out.println("=== 실행 후 ===");
        System.out.println("originList = " + originList);
        System.out.println("resultList = " + resultList);
    }
}

```

```

originList = [ImmutablePerson{name='Kim', age=10}, ImmutablePerson{name='Lee',
age=20}]
=== 실행 후 ===
originList = [ImmutablePerson{name='Kim', age=10}, ImmutablePerson{name='Lee',
age=20}]
resultList = [ImmutablePerson{name='Kim', age=11}, ImmutablePerson{name='Lee',
age=21}]

```

이 예제는 불변 객체(ImmutablePerson)를 사용하므로, 스트림에서 나이를 1씩 증가해도 원본 리스트는 전혀 변경되지 않는다. 결과적으로 새로운 객체 리스트가 생성되어 반환되므로, 부수 효과 없는 안전한 처리가 가능하다.

자바와 함수형 프로그래밍3

일급 시민(First-class Citizen) 함수

- 함수가 일반 값(숫자, 문자열, 객체(자료구조) 등)과 동일한 지위를 가진다.
- 함수를 변수에 대입하거나, 다른 함수의 인자로 전달하거나, 함수에서 함수를 반환하는 고차 함수(Higher-order Function)를 자유롭게 사용할 수 있다.

```
package functional;

import java.util.function.Function;

public class FirstClassCitizenMain {

    public static void main(String[] args) {
        // 함수를 변수에 담는다
        Function<Integer, Integer> func = x -> x * 2;

        // 함수를 인자로 전달
        applyFunction(10, func);

        // 함수를 반환
        getFunc().apply(10);
    }

    // 고차 함수: 함수를 인자로 받음
    public static Integer applyFunction(Integer input, Function<Integer, Integer> func) {
        return func.apply(input);
    }

    // 고차 함수: 함수를 반환
    public static Function<Integer, Integer> getFunc() {
        return x -> x * 2;
    }
}
```

이 예제에서는 자바의 함수형 인터페이스(`Function`)를 사용해 **함수를 변수처럼 취급**하고 있다. 함수를 **인자로 전달**하거나 **반환함**으로써 함수가 일급 시민(First-class Citizen)인 모습을 확인할 수 있다. 이는 고차 함수를 구현하는 기반이 된다.

선언형(Declarative) 접근

- **어떻게**가 아닌 **무엇**을 계산할지 기술한다.
- 복잡한 제어 구조나 상태 관리를 함수의 합성과 함수 호출로 대체하여 간결하고 가독성 높은 코드를 작성한다.

```

package functional;

import java.util.ArrayList;
import java.util.List;

// 짝수면 값을 제공해라
public class DeclarativeMain {

    public static void main(String[] args) {
        List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

        // 명령형: for문과 조건 검사로 처리
        List<Integer> result1 = new ArrayList<>();
        for (int number : numbers) {
            if (number % 2 == 0) { // 짝수인지 확인
                result1.add(number * number); // 제공한 값을 추가
            }
        }
        System.out.println("Imperative Result: " + result1);

        // 선언형: 스트림 API로 처리
        List<Integer> result2 = numbers.stream()
            .filter(number -> number % 2 == 0) // 짝수 필터링
            .map(number -> number * number) // 제곱
            .toList();
        System.out.println("Declarative Result: " + result2);
    }
}

```

실행 결과

```

Imperative Result: [4, 16, 36, 64, 100]
Declarative Result: [4, 16, 36, 64, 100]

```

명령형(Imperative) 방식은 for 문과 조건문으로 **어떻게(How)** 처리할지 구체적으로 작성해야 한다. 반면, **선언형(Declarative)** 방식은 스트림의 `filter`, `map` 같은 함수를 조합해 **무엇(What)**을 할지에 집중한다. 이렇게 선언형으로 작성하면 코드가 더 간결해지고 로직이 명확해진다.

함수 합성(Composition)

- 간단한 함수를 조합해 더 복잡한 함수를 만드는 것을 권장한다.
- 작은 단위의 함수들을 체이닝(Chaining)하거나 파이프라이닝(Pipelining)해서 재사용성을 높이고, 코드 이해도를 높인다.

함수 합성 예제1

```
package functional;

import java.util.function.Function;

public class CompositionMain1 {

    public static void main(String[] args) {
        // 1. x를 입력받아 x * x 하는 함수
        Function<Integer, Integer> square = x -> x * x;
        // 2. x를 입력받아 x + 1 하는 함수
        Function<Integer, Integer> add = x -> x + 1;

        // 함수 합성
        // 1. compose()를 사용한 새로운 함수 생성
        // 먼저 add 적용 후 square 적용하는 새로운 함수 newFunc1 생성
        // square(add(2)) = square(3) = 9
        Function<Integer, Integer> newFunc1 = square.compose(add);
        System.out.println("newFunc1 결과: " + newFunc1.apply(2));

        // 2. andThen()를 사용한 새로운 함수 생성
        // 먼저 square 적용 후 add 적용하는 새로운 함수 newFunc2 생성
        // add(square(2)) = add(4) = 5
        Function<Integer, Integer> newFunc2 = square.andThen(add);
        System.out.println("newFunc2 결과: " + newFunc2.apply(2));
    }
}
```

실행 결과

```
newFunc1 결과: 9
newFunc2 결과: 5
```

- `compose` 는 뒤쪽 함수 → 앞쪽 함수 순으로 적용한다.
- `andThen` 은 앞쪽 함수 → 뒤쪽 함수 순으로 적용한다.
- 이렇게 작은 함수를 여러 개 조합해 새로운 함수를 만들어내는 것이 **함수 합성**이다.

함수 합성 예제2

```
package functional;

import java.util.function.Function;

public class CompositionMain2 {
    public static void main(String[] args) {
        // 1. String -> Integer
        Function<String, Integer> parseInt = Integer::parseInt;

        // 2. Integer -> Integer (여기서는 제곱)
        Function<Integer, Integer> square = x -> x * x;

        // 3. Integer -> String
        Function<Integer, String> toString = x -> "결과: " + x;

        // compose 혹은 andThen으로 합성하기
        // parseInt -> square -> toString 순서로 하고 싶다면 andThen()을 사용
        Function<String, String> finalFunc = parseInt
            .andThen(square)
            .andThen(toString);

        // 문자열 "5"를 입력하면 파싱-> 제곱-> 문자열 출력 순서
        String result1 = finalFunc.apply("5");
        System.out.println(result1); // "결과: 25"

        String result2 = finalFunc.apply("10");
        System.out.println(result2); // "결과: 100"

        // 또 다른 조합으로 사용 가능
        Function<String, Integer> stringToSquareFunc = parseInt
            .andThen(square);
        Integer result3 = stringToSquareFunc.apply("5");
        System.out.println("result3 = " + result3);
    }
}
```

```
}
```

실행 결과

```
결과: 25  
결과: 100  
result3 = 25
```

문자열 "5"를 입력받아 정수로 변환(parseInt) → 제곱(square) → 문자열로 변환(toString)하는 과정을 서로 다른 함수로 분리하고, `andThen`으로 이어 붙여 최종 함수를 구성한다. 이런 식으로 함수를 체이닝(Chaining) 하면, 작은 로직들을 조합해 가독성과 재사용성이 높아진다.

Lazy Evaluation(지연 평가) (선택적 특징)

- 필요한 시점까지 계산을 미루는 평가 전략이다.
- 불필요한 계산 비용을 줄인다.

자바 스트림도 중간 연산(filter, map 등)은 최종 연산(forEach, collect 등)이 호출되기 전까진 실행되지 않는다. 이를 통해 필요한 시점까지 계산을 미루는 전략을 취할 수 있다.

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5);  
Stream<Integer> stream = numbers.stream()  
    .filter(n -> {  
        System.out.println("filter: " + n);  
        return n % 2 == 0;  
    });  
  
// 아직 출력된 것이 없음 (중간 연산만 설정된 상태)  
  
// 최종 연산을 호출할 때 실제 동작 시작  
List<Integer> evens = stream.toList();  
// 여기서야 filter가 실제로 동작하며 콘솔에 filter 로그가 찍힘
```

이처럼 스트림의 중간 연산은 바로 평가되지 않고, 최종 연산이 실행될 때 한 번에 연산이 수행된다. 필요 없는 연산을 미리 실행하지 않으므로, 계산 효율을 높일 수 있다.

정리

1. 프로그래밍 패러다임

- 명령형(How)과 선언형(What)으로 크게 구분된다.
- 명령형 프로그래밍: 절차지향, 객체지향 등을 포함하며, 상태 변화와 제어 흐름을 상세히 기술한다.
- 선언형 프로그래밍: 필요한 결과(무엇)를 선언적으로 표현하고, 내부 구현(어떻게)은 추상화한다. 함수형 프로그래밍 등도 선언형 패러다임에 속한다.

2. 함수형 프로그래밍(Functional Programming)

- 순수 함수(Pure Function), 불변성(Immutable State), 부수 효과(Side Effect) 최소화 등을 핵심으로 하며, 선언형 스타일에 가깝다.
- 함수가 일급 시민(First-class Citizen)으로 취급되어, 함수를 인자로 전달하거나 반환할 수 있는 고차 함수를 자유롭게 활용할 수 있다.
- 이로 인해 병렬 처리나 동시성 프로그래밍에서 유리하며, 코드 가독성과 유지보수성이 높아진다.

3. 자바와 함수형 프로그래밍

- 자바는 객체지향이 주된 패러다임이지만, 동시에 절차지향적인 특징도 부분적으로 활용 가능하다.
- 자바 8 이후로 람다, 함수형 인터페이스(Function, Predicate 등), 스트림 API를 지원하면서 함수형 스타일의 코드 작성이 한층 쉬워졌다.
- 완벽한 함수형 언어는 아니지만, 불변 객체 설계나 순수 함수를 지향하면 자바에서도 함수형 프로그래밍의 이점을 누릴 수 있다.

정리

- 자바는 기본적으로 객체지향 언어이지만, 필요에 따라 절차지향 접근도 가능하고 함수형 요소도 도입되어 있어 **멀티 패러다임** 언어로 발전해왔다.
- 한 가지 패러다임만 고집하기보다는, 프로젝트의 목적과 상황에 따라 **절차지향**, **객체지향**, **함수형** 스타일을 조합해 사용하는 것이 실제 개발에서 더 효과적이다.
- 특히 함수형 프로그래밍에서 강조하는 순수 함수, 불변성, 부수 효과 최소화 개념은 **코드를 단순화**하고 **동시성 문제를 줄이는 데** 큰 도움이 되므로, 자바를 사용하는 개발자라면 적극적으로 활용할 가치가 있다.